

Вызов функций C/C++ на NASM

Microsoft Windows ABI

При обращении к функциями C/C++ на Windows в коде ассемблера NASM необходимо следовать некоторым соглашениям, которые известны как **Microsoft Windows ABI** (Application Binary Interface). Эти соглашения определяют, как данные передаются в функции и как результат возвращается из функции.

Microsoft Windows ABI

Вызывающий код передает первые четыре параметра в регистры. Для передачи в функцию первых четырех параметров (целочисленных) используются регистры RCX, RDX, R8 и R9 соответственно. Если параметры представляют числа с плавающей точкой, то они передаются через регистры XMM0, XMM1, XMM2 и XMM3 и дублируются в соответствующих регистрах RCX/RDX/R8/R9. То есть например, если второй параметр представляет число с плавающей точкой, то оно помещается в XMM1 и дублируется в RDX.

Microsoft Windows ABI

Параметры начиная с 5-го передаются через стек. Так, 5-й параметр должен занимать в стеке место по адресу $RSP+32$, 6-й параметр - по адресу $RSP+40$ и так далее.

Microsoft Windows ABI

Если процедура имеет смешанные параметры - одновременно и целочисленные, и с плавающей запятой, то каждый параметр помещается в соответствующий его номеру регистр RCX/RDX/R8/R9 или XMM0/XMM1/XMM2/XMM3.

Microsoft Windows ABI

Пример:

```
void someFunc(int a, double b, char *c, double d)
```

Параметры будут размещаться следующим образом

RCX	a
XMM1 и RDX	b
R8	c
XMM3 и R9	d

Microsoft Windows ABI

- Параметры всегда представляют собой 8-байтовые значения.
- Вызывающий код должен зарезервировать для параметров в стеке как минимум 32 байта (**shadow storage**)
- Условно можно сказать, что первый параметр занимает в стеке память по адресу RSP, второй - по адресу RSP+8, третий - RSP+16, четвертый - RSP+24
- Указатель стека RSP должен быть выровнен по 16 байтам непосредственно перед тем, как инструкция call поместит адрес возврата в стек.
- Если функция возвращает целое число, то оно помещается в регистр RAX, если число с плавающей точкой - то в регистр XMM0.

Точка входа в программу

При использовании функций C/C++ в программе на ассемблере в качестве точки входа используют метку **main** (а не **_start**):

```
global main    ; функция main - точка входа
section .text
main:
    ; инструкции функции main
    ret
```

Кроме того, программа теперь завершается инструкцией **ret**, как любая обычная функция

Вызов функций C/C++ на Windows

```
global main ; функция main - точка входа
```

```
extern printf ; подключаем функцию printf
```

```
section .data
```

```
message db "Hello world",10 ; строка для вывода
```

```
section .text
```

```
main:
```

```
    sub rsp, 40 ; выравнивание должно быть по 16 байтам с учетом 32 байтов - shadows storage  
функции
```

```
    lea rcx, [rel message] ; первый параметр функции printf - строка для вывода на консоль
```

```
    call printf
```

```
    add rsp, 40 ; восстанавливаем стек
```

```
    ret
```

Передача параметров в функцию C/C++

global main ; функция main - точка входа

extern printf ; подключаем функцию

section .data

message db "Name: %s Age: %u Company: %s Salary: %u",10,0 ; строка для вывода

name db "Tom",0

age dq 39

company db "METANIT.COM",0

salary dq 1150

section .text

main:

sub rsp, 40 ; выравнивание должно быть по 16 байтам

lea rcx, [rel message] ; первый параметр функции printf - строка форматирования

lea rdx, [rel name] ; первый аргумент для строки форматирования

mov r8, [rel age] ; второй аргумент для строки форматирования

lea r9, [rel company] ; третий аргумент для строки форматирования

mov r10, [rel salary]

mov qword [rsp+32], r10 ; четвертый аргумент для строки форматирования

call printf

add rsp, 40 ; восстанавливаем стек

ret

Вызов функций NASM на C/C++ +

Вызов функции ассемблера из C/C++

Как код ассемблера может вызывать функции на C/C++, так и код на C/C++ может использовать функции ассемблера. И в данном случае действуют те же условности по параметрам и возвращаемым результатам.

Windows

global sum

section .text

; Функция возвращает сумму двух чисел

; Принимает два параметра:

; rcx - первое число

; rdx - второе число

; Результат функции возвращается через регистр rax

sum:

mov rax, rcx

add rax, rdx

ret

Компиляция

C:\Program Files (x86)\SASM\NASM

```
nasm -f win64 sum.asm -o sum.obj
```

Файл **app.c**

```
#include <stdio.h>
```

```
int sum(int, int); // прототип функции из файла на ассемблере
```

```
int main(void) {  
    int result = sum(4, 2); // вызываем функцию  
    printf("result = %d\n", result);  
}
```

C:\Program Files (x86)\SASM\MinGW64\bin

```
gcc app.c sum.obj -o app.exe
```

Или под Visual Studio

```
cl app.c sum.obj
```

Компиляция

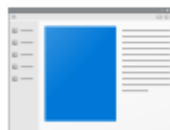
```
C:\Users\221\Students>cl 1.c sum.obj
Оптимизирующий компилятор Microsoft (R) C/C++ версии 19.50.35725 для x64
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

1.c
Microsoft (R) Incremental Linker Version 14.50.35725.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:1.exe
1.obj
sum.obj

C:\Users\221\Students>1.exe
result = 6

C:\Users\221\Students>
```



1.exe



1.obj



sum.obj



2.obj



1.asm

Message Box

- [NASM Message Box 64 bit](#)